

```

import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
from sklearn.utils import shuffle
import matplotlib.image as mpimg
from imgaug import augmenters as iaa
import cv2
import random

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Convolution2D, Flatten, Dense
from tensorflow.keras.optimizers import Adam

def getName(filepath):
    return filepath.split('\\')[-1]

def importDatainfo(path):
    coloumns = ['Center', 'Left', 'Right', 'Steering', 'Throttle', 'Brake', 'Speed']
    data = pd.read_csv(os.path.join(path, 'driving_log.csv'), names = coloumns)
    #print(data.head())
    #print(data['Center'][0])
    #print(getName(data['Center'][0]))
    data['Center'] = data['Center'].apply(getName)
    #print(data.head())
    print('Total Images Imported: ', data.shape[0])
    return data

def balanceData(data, display = True):
    nBins = 31
    samplesPerBin = 1000
    hist, bins = np.histogram(data['Steering'], nBins)
    #print(bins)
    if display:
        center = (bins[:-1] + bins[1:])*0.5
        #print(center)
        plt.bar(center, hist, width = 0.06)
        plt.plot((-1, 1), (samplesPerBin, samplesPerBin))
        plt.show()

    removeIndexList = []
    for j in range(nBins):
        binDataList = []
        for i in range(len(data['Steering'])):
            if data['Steering'][i] >= bins[j] and data['Steering'][i] <= bins[j+1]:
                binDataList.append(i)
        binDataList = shuffle(binDataList)
        binDataList = binDataList[samplesPerBin:]
        removeIndexList.extend(binDataList)
    print('Removed Images: ', len(removeIndexList))
    data.drop(data.index[removeIndexList], inplace = True)
    print('Remaining Images: ', len(data))

    if display:
        hist, _ = np.histogram(data['Steering'], nBins)
        plt.bar(center, hist, width = 0.06)
        plt.plot((-1, 1), (samplesPerBin, samplesPerBin))
        plt.show()

    return data

def loadData(path, data):
    imagesPath = []
    steering = []

    for i in range(len(data)):
        indexedData = data.iloc[i]
        #print(indexedData)
        imagesPath.append(os.path.join(path, 'IMG', indexedData[0]))
        steering.append(float(indexedData[3]))

    imagesPath = np.asarray(imagesPath)
    steering = np.asarray(steering)
    return imagesPath, steering

def augmentImage(imgPath, steering):
    img = mpimg.imread(imgPath)

```

```
img = mpimg.imread(imgPath)
#print(np.random.rand(),np.random.rand(),np.random.rand(),np.random.rand())
## PAN
if np.random.rand()<0.5:
    pan = iaa.Affine(translate_percent = {'x':(-0.1,0.1),'y':(-0.1,0.1)})
    img = pan.augment_image(img)
## ZOOM
if np.random.rand()<0.5:
    zoom = iaa.Affine(scale =(1,1.2))
    img = zoom.augment_image(img)
## BRIGHTNESS
if np.random.rand()<0.5:
    brightness =iaa.Multiply((0.4,1.2))
    img = brightness.augment_image(img)
## FLIP
if np.random.rand()<0.5:
    img = cv2.flip(img,1)
    steering = -steering

return img,steering

#imgRe, st = augmentImage('test.jpg',0)
#plt.imshow(imgRe)
#plt.show()

def preProcessing(img):
    img = img[60:135,:,:]
    img = cv2.cvtColor(img, cv2.COLOR_RGB2YUV)
    img = cv2.GaussianBlur(img,(3,3),0)
    img = cv2.resize(img,(200,66))
    img = img /255
    return img

imgRe = preProcessing(mpimg.imread(' /content/drive/MyDrive/Colab Notebooks/test.jpg'))
plt.imshow(imgRe)
plt.show()

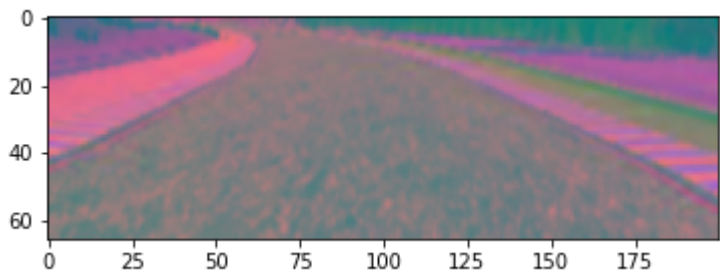
def batchGen(imagesPath,steeringList,batchSize,trainFlag):
    while True:
        imgBatch = []
        steeringBatch = []

        for i in range(batchSize):
            index = random.randint(0,len(imagesPath)-1)
            if trainFlag:
                img, steering =augmentImage(imagesPath[index], steeringList[index])
            else:
                img = mpimg.imread(imagesPath[index])
                steering = steeringList[index]

            img =preProcessing(img)
            imgBatch.append(img)
            steeringBatch.append(steering)
        yield(np.asarray(imgBatch),np.asarray(steeringBatch))

def creatModel():
    model = Sequential()
    model.add(Convolution2D(24,(5,5),(2,2),input_shape = (66,200,3),activation = 'elu'))
    model.add(Convolution2D(36,(5,5),(2,2),activation = 'elu'))
    model.add(Convolution2D(48,(5,5),(2,2),activation = 'elu'))
    model.add(Convolution2D(64,(3,3),activation = 'elu'))
    model.add(Convolution2D(64,(3,3),activation = 'elu'))

    model.add(Flatten())
    model.add(Dense(100,activation='elu'))
    model.add(Dense(50,activation='elu'))
    model.add(Dense(10,activation='elu'))
    model.add(Dense(1))
    opt = Adam(learning_rate=0.001 )
    model.compile(loss ='mse',optimizer = opt)
    return model
```



```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Training

```
print('Setting Up')
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
#from utlis import *
from sklearn.model_selection import train_test_split

#### STEP 1
path = '/content/drive/MyDrive/Colab Notebooks/myData1'
data = importDatainfo(path)

#### STEP 2
data = balanceData(data,display = True)

#### STEP 3
imagesPath, steerings = loadData(path,data)
#print(imagesPath[0],steering[0])

#### STEP 4
xTrain, xVal, yTrain, yVal = train_test_split(imagesPath,steerings,test_size = 0.2, random_state = 5)
print('Total Training Images: ', len(xTrain))
print('Total validation Images: ', len(xVal))

#### STEP 5

#### STEP 6

#### STEP 7

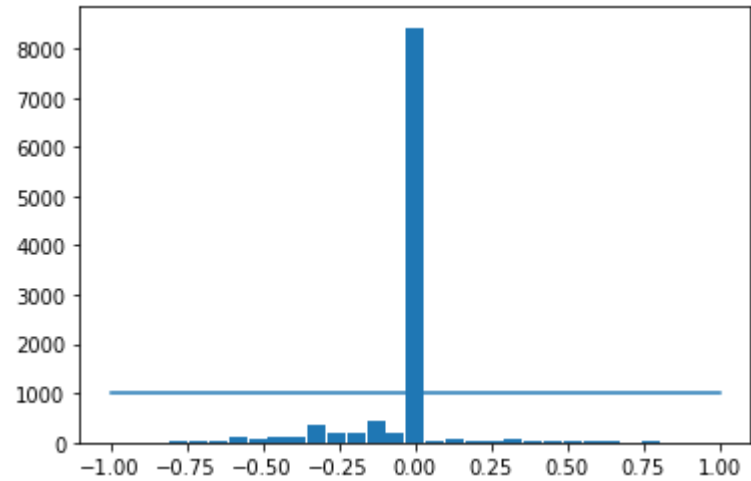
#### STEP 8
model=creatModel()
model.summary()

#### STEP 9
history = model.fit(batchGen(xTrain,yTrain,40,1),steps_per_epoch = 300,epochs = 50,validation_data = batchGen(xVal,yVal,40,0))

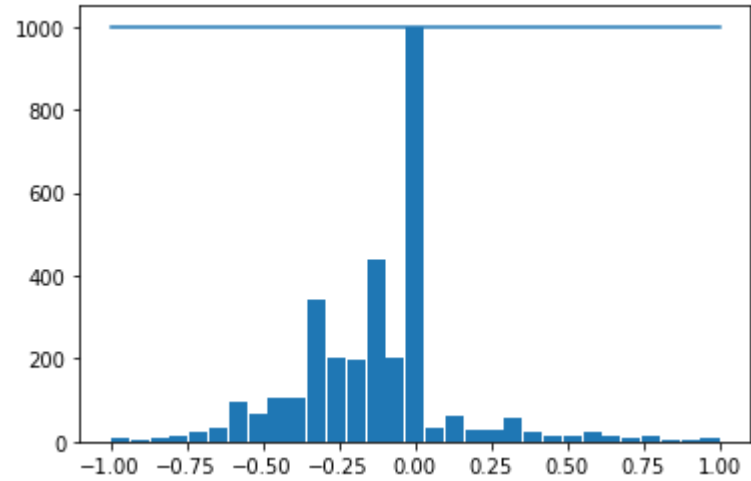
#### STEP 10
model.save('model7.h5')
print('Model Saved')

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.legend(['Training','Validation'])
plt.ylim([0,1])
plt.title('Loss')
plt.xlabel('Epoch')
plt.show()
```

Setting Up
Total Images Imported: 10615



Removed Images: 7422
Remaining Images: 3193



Total Training Images: 2554
Total validation Images: 639
Model: "sequential_2"

Layer (type)	Output Shape	Param #
=====		
conv2d_10 (Conv2D)	(None, 31, 98, 24)	1824
conv2d_11 (Conv2D)	(None, 14, 47, 36)	21636
conv2d_12 (Conv2D)	(None, 5, 22, 48)	43248
conv2d_13 (Conv2D)	(None, 3, 20, 64)	27712
conv2d_14 (Conv2D)	(None, 1, 18, 64)	36928
flatten_2 (Flatten)	(None, 1152)	0
dense_8 (Dense)	(None, 100)	115300
dense_9 (Dense)	(None, 50)	5050
dense_10 (Dense)	(None, 10)	510
dense_11 (Dense)	(None, 1)	11
=====		

Total params: 252,219
Trainable params: 252,219
Non-trainable params: 0

Epoch 1/50
300/300 [=====] - 302s 1s/step - loss: 0.1148 - val_loss: 0.0930
Epoch 2/50
300/300 [=====] - 93s 310ms/step - loss: 0.0841 - val_loss: 0.1128
Epoch 3/50
300/300 [=====] - 91s 304ms/step - loss: 0.0754 - val_loss: 0.0518
Epoch 4/50
300/300 [=====] - 90s 301ms/step - loss: 0.0668 - val_loss: 0.0459
Epoch 5/50
300/300 [=====] - 90s 302ms/step - loss: 0.0643 - val_loss: 0.0393
Epoch 6/50
300/300 [=====] - 89s 299ms/step - loss: 0.0715 - val_loss: 0.0641
Epoch 7/50
300/300 [=====] - 93s 312ms/step - loss: 0.0912 - val_loss: 0.0765
Epoch 8/50
300/300 [=====] - 93s 309ms/step - loss: 0.0889 - val_loss: 0.0718
Epoch 9/50
300/300 [=====] - 90s 300ms/step - loss: 0.0844 - val_loss: 0.0623
Epoch 10/50
300/300 [=====] - 90s 302ms/step - loss: 0.0938 - val_loss: 0.0856
Epoch 11/50
300/300 [=====] - 91s 303ms/step - loss: 0.0900 - val_loss: 0.0741
Epoch 12/50
300/300 [=====] - 91s 306ms/step - loss: 0.0903 - val_loss: 0.0769
Epoch 13/50
300/300 [=====] - 90s 301ms/step - loss: 0.0877 - val_loss: 0.0711
Epoch 14/50
300/300 [=====] - 90s 302ms/step - loss: 0.0900 - val_loss: 0.0691

Epoch 15/50
300/300 [=====] - 94s 315ms/step - loss: 0.0878 - val_loss: 0.0787
Epoch 16/50
300/300 [=====] - 91s 303ms/step - loss: 0.0869 - val_loss: 0.0720
Epoch 17/50
300/300 [=====] - 90s 301ms/step - loss: 0.0877 - val_loss: 0.0637
Epoch 18/50
300/300 [=====] - 90s 300ms/step - loss: 0.0889 - val_loss: 0.0666
Epoch 19/50
300/300 [=====] - 89s 298ms/step - loss: 0.0856 - val_loss: 0.0748
Epoch 20/50
300/300 [=====] - 89s 299ms/step - loss: 0.0884 - val_loss: 0.0806
Epoch 21/50
300/300 [=====] - 90s 302ms/step - loss: 0.0870 - val_loss: 0.0929
Epoch 22/50
300/300 [=====] - 89s 299ms/step - loss: 0.0866 - val_loss: 0.0768
Epoch 23/50
300/300 [=====] - 89s 298ms/step - loss: 0.0874 - val_loss: 0.0756
Epoch 24/50
300/300 [=====] - 88s 296ms/step - loss: 0.0876 - val_loss: 0.0659
Epoch 25/50
300/300 [=====] - 88s 295ms/step - loss: 0.0886 - val_loss: 0.0744
Epoch 26/50
300/300 [=====] - 88s 293ms/step - loss: 0.0892 - val_loss: 0.0720
Epoch 27/50
300/300 [=====] - 88s 294ms/step - loss: 0.0886 - val_loss: 0.0690
Epoch 28/50
300/300 [=====] - 88s 293ms/step - loss: 0.0876 - val_loss: 0.0706
Epoch 29/50
300/300 [=====] - 88s 295ms/step - loss: 0.0894 - val_loss: 0.0703
Epoch 30/50
300/300 [=====] - 88s 293ms/step - loss: 0.0894 - val_loss: 0.0789
Epoch 31/50
300/300 [=====] - 88s 294ms/step - loss: 0.0913 - val_loss: 0.0766
Epoch 32/50
300/300 [=====] - 88s 293ms/step - loss: 0.0872 - val_loss: 0.0692
Epoch 33/50
300/300 [=====] - 87s 291ms/step - loss: 0.0851 - val_loss: 0.0705
Epoch 34/50
300/300 [=====] - 87s 290ms/step - loss: 0.0879 - val_loss: 0.0730
Epoch 35/50
300/300 [=====] - 86s 289ms/step - loss: 0.0889 - val_loss: 0.0713
Epoch 36/50
300/300 [=====] - 88s 294ms/step - loss: 0.0850 - val_loss: 0.0861
Epoch 37/50
300/300 [=====] - 87s 291ms/step - loss: 0.0857 - val_loss: 0.0744
Epoch 38/50
300/300 [=====] - 87s 291ms/step - loss: 0.0856 - val_loss: 0.0759
Epoch 39/50
300/300 [=====] - 87s 292ms/step - loss: 0.0886 - val_loss: 0.0707
Epoch 40/50
300/300 [=====] - 88s 295ms/step - loss: 0.0863 - val_loss: 0.0788
Epoch 41/50
300/300 [=====] - 87s 293ms/step - loss: 0.0849 - val_loss: 0.0679
Epoch 42/50
300/300 [=====] - 89s 297ms/step - loss: 0.0901 - val_loss: 0.0732
Epoch 43/50
300/300 [=====] - 89s 296ms/step - loss: 0.0848 - val_loss: 0.0726
Epoch 44/50
300/300 [=====] - 88s 295ms/step - loss: 0.0887 - val_loss: 0.0739
Epoch 45/50
300/300 [=====] - 89s 298ms/step - loss: 0.0877 - val_loss: 0.0728
Epoch 46/50
300/300 [=====] - 89s 298ms/step - loss: 0.0859 - val_loss: 0.0762
Epoch 47/50
300/300 [=====] - 89s 296ms/step - loss: 0.0848 - val_loss: 0.0721
Epoch 48/50
300/300 [=====] - 88s 295ms/step - loss: 0.0843 - val_loss: 0.0776
Epoch 49/50
300/300 [=====] - 89s 296ms/step - loss: 0.0918 - val_loss: 0.0722
Epoch 50/50
300/300 [=====] - 88s 295ms/step - loss: 0.0852 - val_loss: 0.0791
Model Saved

